

Contents

1	Algoritmos, complejidad y estrategias de resolución	1
1.1	La receta, la despensa y la escala biológica	1
1.1.1	Niveles de representación de un algoritmo	2
1.2	El coste de un algoritmo y el modelo de computación	2
1.2.1	¿Por qué no medimos el coste en segundos?	3
1.2.2	Definición del tamaño de entrada en problemas multivariantes	3
1.2.3	El modelo RAM de operaciones elementales	3
1.2.4	Conteo de operaciones en tres algoritmos de ejemplo	4
1.3	Notación Asintótica: el lenguaje de la eficiencia	5
1.3.1	Las tres reglas de simplificación de la notación Big-O	5
1.3.2	Escala de complejidades comunes	5
1.3.3	Cotas superior, inferior y ajustada	6
1.3.4	Peor caso, mejor caso y caso promedio	7
1.3.5	Clasificación de patrones elementales	7
1.3.6	Demostración matemática del término dominante	7
1.4	Las cinco grandes familias de diseño algorítmico	8
1.4.1	1. Fuerza bruta y búsqueda exhaustiva	8
1.4.2	2. Algoritmos voraces (<i>Greedy</i>)	8
1.4.3	3. Divide y vencerás y recursión	10
1.4.4	4. Programación dinámica	11
1.4.5	5. Vuelta atrás (<i>Backtracking</i>)	12
1.5	La operación peligrosa: la explosión combinatoria oculta	13
1.6	Actividad de depuración: localizar fallos en algoritmos ajenos	13
1.7	Síntesis y diagnóstico de problemas algorítmicos	14
1.7.1	Tabla de diagnóstico de síntomas computacionales	14
1.7.2	Tabla de síntesis operativa con preguntas de control	15
1.8	Glosario esencial	15
1.9	Conexión con el curso y lo que viene después	16
1.10	Fuentes y límites de uso	16
	Anexo práctico · Auditar el coste y sentir la diferencia entre exponencial y lineal	17

Chapter 1

Algoritmos, complejidad y estrategias de resolución

1.1 La receta, la despensa y la escala biológica

En los dos primeros capítulos aprendimos a desenvolvernó en la terminal de Linux y a construir procedimientos automáticos con Bash, tuberías y expresiones regulares. Con esas herramientas podemos procesar archivos de datos, filtrar registros biológicos y encadenar programas. Sin embargo, en cuanto pasamos de pequeños ficheros de prueba a conjuntos de datos biomédicos a gran escala, surge de inmediato un obstáculo insalvable si solo aplicamos fuerza bruta: el **coste computacional**.

Consideremos un ejemplo real en genómica: el genoma humano contiene aproximadamente 3×10^9 pares de bases (3000 millones de nucleótidos). Si un secuenciador de última generación nos entrega 100 millones de lecturas cortas (*FASTQ*) y diseñamos un procedimiento ingenuo que compare cada lectura posición a posición contra todo el genoma desde el principio, el ordenador tendría que realizar del orden de $100 \times 10^6 \times 3 \times 10^9 = 3 \times 10^{17}$ operaciones. Incluso en un superordenador capaz de ejecutar mil millones de operaciones por segundo, ese cálculo ingenuo tardaría más de **9 años de computación ininterrumpida**.

El cálculo ilustra por qué la fuerza bruta es inviable a escala genómica. Las herramientas bioinformáticas reales no comparan cada lectura contra cada posición: construyen o reutilizan índices estructurados (como tablas de dispersión, árboles de sufijos o la transformada de Burrows-Wheeler, que estudiaremos en los siguientes capítulos) que descartan instantáneamente la inmensa mayoría del espacio de búsqueda. La diferencia entre un cálculo inviable que tardaría años y un análisis rápido no reside en comprar un ordenador más caro, sino en la **eficiencia matemática del algoritmo**.

Si una estructura de datos es la **despensa** —la forma en que se organizan y almacenan los ingredientes en la memoria—, un algoritmo es la **receta**: la secuencia ordenada de instrucciones que transforma los datos de entrada en el resultado biológico deseado. Este capítulo se centra en la receta apoyándose en lo que ya sabemos representar en Bash y pseudocódigo (TC-CH01 y TC-CH02). Las estructuras de datos avanzadas se formalizarán en los capítulos TC-CH04 y TC-CH05; aquí nos basta con saber que las entradas y salidas de un algoritmo son datos, independientemente de dónde y cómo se guarden.

Hoy en día, cualquier asistente de inteligencia artificial puede generar código en segundos, pero con frecuencia elige la estrategia equivocada: fuerza bruta donde existía una solución elegante, o un método voraz que parece resolver el problema pero no garantiza la respuesta óptima global. Reconocer el paradigma que tenemos delante, estimar su coste antes de lanzarlo y juzgar si escalará a nuestros datos es la competencia fundamental que proporciona este capítulo.

Un algoritmo es el procedimiento abstracto que resuelve un problema; un programa es su realización concreta en un lenguaje de programación. El mismo algoritmo puede escribirse como programas distintos en Bash o cualquier otro lenguaje de programación.

Un algoritmo es una secuencia finita de pasos bien definidos que, a partir de una entrada, produce una salida en un tiempo finito. «Finita» y «bien definidos» son las palabras importantes: un algoritmo no puede ser ambiguo ni eterno.

1.1.1 Niveles de representación de un algoritmo

Antes de lanzarse a teclear código en el ordenador, un algoritmo debe diseñarse y evaluarse conceptualmente. Para ello se utilizan cuatro niveles de representación, ordenados de menor a mayor rigor y precisión:

1. **Lenguaje natural:** describe la idea en lenguaje humano ordinario («recorrer la lista y guardar el número mayor»). Es muy intuitivo para comunicar la idea inicial, pero es propenso a ambigüedades e imprecisiones.
2. **Diagrama de flujo:** representación gráfica normalizada que utiliza cajas y flechas para ilustrar decisiones lógicas, bucles y flujos de datos.
3. **Pseudocódigo:** el punto dulce de la ingeniería. Es un lenguaje estructurado en texto, libre de las rigideces sintácticas de un lenguaje de programación concreto, pero con suficiente rigor lógico para analizar el coste y la corrección del algoritmo. Es la forma estándar en la que razonaremos los algoritmos a lo largo de este libro.
4. **Lenguaje de programación:** código ejecutable (Bash, C u otros lenguajes estructurados) que el sistema operativo puede ejecutar directamente.

Los cuatro niveles de representación de un algoritmo, de menor a mayor precisión, son: lenguaje natural, diagrama de flujo, pseudocódigo y lenguaje de programación.

La ruta **esencial** comprende la definición de algoritmo, el modelo de coste en operaciones elementales, las reglas de la notación Big-O (O), los órdenes de complejidad estándar y las cinco grandes familias de diseño (fuerza bruta, voraz, divide y vencerás, programación dinámica y backtracking); 110–130 minutos. La ruta de **estudio** profundiza en el análisis formal con cotas Ω y Θ , la demostración matemática del término dominante, la comparación rigurosa entre recursión e iteración, la memoización ascendente y la actividad de auditoría práctica de costes; 60–80 minutos adicionales.

1.2 El coste de un algoritmo y el modelo de computación

Todo algoritmo, al ejecutarse en un ordenador real, consume dos **recursos fundamentales**:

- **Tiempo de ejecución:** medido en ciclos de procesamiento de la unidad central de procesamiento (CPU).
- **Espacio de memoria:** medido en la cantidad de memoria principal (RAM) adicional requerida para almacenar variables, tablas y estructuras auxiliares.

Un algoritmo consume dos recursos al ejecutarse: tiempo (ciclos de CPU) y espacio (memoria adicional que necesita).

1.2.1 ¿Por qué no medimos el coste en segundos?

Una tentación habitual en los primeros pasos computacionales es medir el tiempo que tarda un programa usando un cronómetro (con la orden `time`). Aunque la medición del tiempo de reloj (*wall-clock time*) es útil para comprobar el rendimiento práctico, **no sirve como medida teórica del algoritmo en sí**.

Si ejecutamos el mismo programa en un portátil antiguo, en un servidor de cómputo moderno, en una máquina con poca memoria o con el sistema operativo ocupado en otras tareas, el tiempo en segundos variará enormemente. Del mismo modo, el tiempo dependerá de si el código fue escrito en un lenguaje compilado de bajo nivel como C o en un lenguaje interpretado.

Para obtener una medida intrínseca y universal del algoritmo, la informática teórica utiliza el **conteo de operaciones elementales** en función del tamaño de los datos de entrada, habitualmente denotado por la variable n .

El coste de un algoritmo se mide contando operaciones elementales en función del tamaño de la entrada (n) en vez de en segundos, porque el tiempo en segundos depende de la máquina, la carga del sistema y el lenguaje, no solo del algoritmo.

1.2.2 Definición del tamaño de entrada en problemas multivariantes

En los ejemplos introductorios solemos resumir el tamaño de la entrada en una única variable n (como el número de elementos de una lista). Sin embargo, en biología computacional la entrada es habitualmente **multivariable**:

- En análisis de secuencias: M lecturas de longitud L cada una (la entrada total tiene tamaño $M \times L$).
- En alineamiento por pares: dos secuencias de longitudes n y m (la matriz de programación dinámica tiene orden $O(n \cdot m)$).
- En redes y vías metabólicas: un grafo con $|V|$ nodos (genes/metabolitos) y $|E|$ interacciones (aristas).

Antes de calcular cualquier complejidad, la primera pregunta obligatoria del científico es: *¿qué parámetros describen cuantitativamente el tamaño de los datos de entrada?*

1.2.3 El modelo RAM de operaciones elementales

Esta cuantificación se apoya en el modelo clásico de la máquina de acceso aleatorio (*Random Access Machine* o modelo RAM):

- La **memoria** se modela como un vector continuo de celdas con dirección propia donde leer o escribir datos toma tiempo constante ($O(1)$).
- La **CPU** ejecuta instrucciones elementales (sumas, restas, multiplicaciones, comparaciones lógicas, asignaciones y saltos de control) sobre palabras de tamaño fijo. Cada una de estas operaciones elementales se contabiliza con un coste unitario constante.

En el modelo RAM, la memoria es una sucesión de direcciones y la CPU ejecuta operaciones aritméticas, lógicas, de entrada/salida y de control sobre palabras de tamaño fijo; cada una de esas operaciones se cuenta como de coste constante.

1.2.4 Conteo de operaciones en tres algoritmos de ejemplo

Para comprender de forma práctica cómo se cuentan las operaciones elementales, analicemos dos algoritmos en Bash que resuelven exactamente la misma tarea: mostrar los N primeros múltiplos de 3.

Listing 1.1: Dos algoritmos para mostrar los N primeros múltiplos de 3. Se reproducen y cuentan sus operaciones con `verification/chapter_results.sh`.

```
# Algoritmo A: un bucle simple directo
for ((i = 1; i <= N; i++)); do
    echo $((i * 3))
done

# Algoritmo B: bucles anidados con sumas acumuladas
for ((i = 1; i <= N; i++)); do
    sum=0
    for ((j = 1; j <= i; j++)); do
        sum=$((sum + 3))
    done
    echo "$sum"
done
```

Desglose del Algoritmo A (Lineal):

1. Inicialización: $i=1$ (1 operación).
2. En cada una de las N iteraciones:
 - Comparación de control: $i \leq N$ (1 op).
 - Multiplicación aritmética: $i * 3$ (1 op).
 - Emisión por salida estándar: `echo` (1 op).
 - Incremento y asignación: $i++$ (2 ops).

Total por vuelta: 5 operaciones.

3. Comparación final que finaliza el bucle: 1 operación.

El número total de operaciones elementales es $T_A(N) = 1 + 5N + 1 \approx 5N + 2$. El número de operaciones crece de forma estrictamente **lineal** con respecto a N : si multiplicamos N por 10, el tiempo se multiplica por 10.

El algoritmo A hace una operación de comparación, una de multiplicación y una de impresión por cada una de las N vueltas del bucle, más el establecimiento inicial de la variable: del orden de $1 + 5N$ operaciones, $O(N)$. Se reproduce con `verification/chapter_results.sh classify_single`.

Desglose del Algoritmo B (Cuadrático): En el Algoritmo B, el bucle interior se ejecuta 1 vez en la primera iteración, 2 veces en la segunda, 3 en la tercera, y así sucesivamente hasta N veces en la última.

El número total de ejecuciones del cuerpo interior viene dado por la célebre suma de Gauss:

$$\sum_{i=1}^N i = 1 + 2 + 3 + \cdots + N = \frac{N(N+1)}{2} = \frac{1}{2}N^2 + \frac{1}{2}N$$

El coste total de este algoritmo incluye un término en N^2 . A diferencia del Algoritmo A, si multiplicamos N por 10 (de $N = 100$ a $N = 1000$), el tiempo de ejecución se multiplica aproximadamente por **100**.

El algoritmo B ejecuta su bucle interior i veces en cada una de las N vueltas del bucle exterior: el total de vueltas del bucle interior es $1 + 2 + \dots + N = N(N + 1)/2$, del orden de N^2 , $O(N^2)$. Se reproduce con `verification/chapter_results.sh classify_nested`.

Alto y comprueba (2 minutos). Prediga cuántas operaciones internas ejecutará el bucle interior del Algoritmo B si $N = 10$. Calcule: $\frac{10 \times 11}{2} = 55$. Ejecute en su terminal: `bash verification/chapter_results.sh classify_nested 10`. ¿Coincide la salida con su predicción?

1.3 Notación Asintótica: el lenguaje de la eficiencia

Al analizar algoritmos, los detalles menores de implementación (como si un bucle hace $5N + 2$ o $3N + 7$ operaciones) carecen de importancia cuando los datos crecen hacia millones de elementos. Lo que determina si un pipeline bioinformático es viable o inviable es su **tasa de crecimiento asintótico**.

La **notación Big-O** (O) es el estándar matemático universal para describir cómo escala el tiempo o la memoria de un algoritmo cuando el tamaño de entrada $n \rightarrow \infty$.

1.3.1 Las tres reglas de simplificación de la notación Big-O

1. **Conservar el término dominante:** si una función de coste tiene múltiples términos ($f(n) = 3n^2 + 500n + 10\,000$), para valores grandes de n el término cuadrático n^2 eclipsa totalmente a los demás.
2. **Descartar constantes multiplicativas:** $5n$ y $100n$ pertenecen ambos a la clase $O(n)$, ya que las constantes dependen de la velocidad de la CPU o del compilador, no del comportamiento intrínseco del algoritmo.
3. **Describir el comportamiento asintótico:** evaluar el límite cuando n crece sin cota.

La notación O sigue tres reglas: conservar el término dominante, descartar las constantes multiplicativas y describir el comportamiento asintótico (la tendencia cuando n crece sin límite).

1.3.2 Escala de complejidades comunes

Notación	Nombre	Efecto al duplicar n	Ejemplo biológico
$O(1)$	Constante	No cambia	Acceso a base por índice
$O(\log n)$	Logarítmica	Suma una operación constante	Búsqueda binaria en tabla
$O(n)$	Lineal	Se duplica ($2\times$)	Conteo de GC en secuencia
$O(n \log n)$	Cuasilineal	Algo más del doble ($2.2\times$)	Ordenación con Mergesort
$O(n^2)$	Cuadrática	Se cuadruplica ($4\times$)	Comparación de pares (todas con todas)
$O(2^n)$	Exponencial	Se eleva al cuadrado	Ensamblaje ingenuo sin poder
$O(n!)$	Factorial	Explosión combinatoria	Alineamiento múltiple por fuerza bruta

Con $n = 1\,000\,000$, un algoritmo $O(n)$ hace un millón de pasos, uno $O(n^2)$ hace un billón, y uno $O(2^n)$ es directamente irrealizable.

El punto de cruce: constante oculta frente a tasa de crecimiento. Un error conceptual habitual es suponer que un algoritmo $O(n)$ es siempre más rápido que uno $O(n^2)$ para cualquier tamaño de entrada n . La notación asintótica describe la tendencia cuando $n \rightarrow \infty$, no el tiempo absoluto sobre entradas pequeñas.

Consideremos dos algoritmos con tiempos de trabajo abstractos $T_A(n) = 50n$ (lineal con constante alta) y $T_B(n) = n^2$ (cuadrático con constante baja):

- Para $n = 20$: $T_A(20) = 50 \times 20 = 1000$, mientras que $T_B(20) = 20^2 = 400$. Para entradas pequeñas, el algoritmo cuadrático realiza menos trabajo.
- Para $n = 50$: $T_A(50) = 2500$ y $T_B(50) = 2500$ (punto de cruce).
- Para $n = 1000$: $T_A(1000) = 50\,000$, mientras que $T_B(1000) = 1\,000\,000$. A partir del punto de cruce, la tasa cuadrática domina y su desventaja crece sin límite.

Por esta razón, las librerías científicas optimizadas combinan con frecuencia algoritmos cuadráticos sencillos para casos diminutos y conmutan a algoritmos asintóticamente superiores cuando los datos crecen (estrategia híbrida que estudiaremos en TC-CH06).

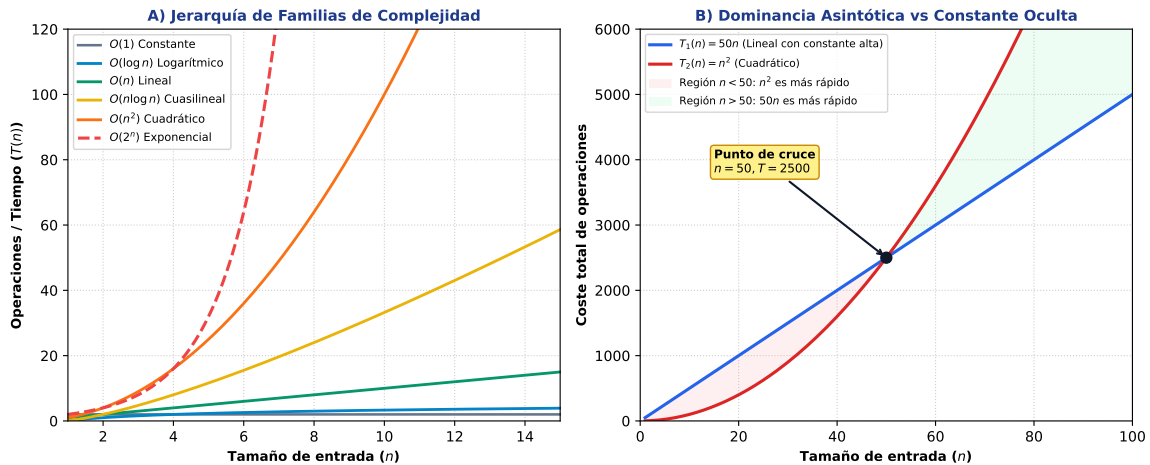


Figure 1.1: A) Jerarquía de crecimiento de las principales familias asintóticas. B) Detalle analítico del punto de cruce en $n = 50$ entre $T_1(n) = 50n$ y $T_2(n) = n^2$, ilustrando por qué la constante oculta domina para entradas pequeñas ($n < 50$) pero el orden de complejidad domina para problemas a escala genómica ($n \gg 50$). Figura original adaptada de las presentaciones docentes.

1.3.3 Cotas superior (O), inferior (Ω) y ajustada (Θ)

En el análisis de algoritmos formal se distinguen tres cotas asintóticas fundamentales:

- **Cota superior $O(g(n))$ (Big-O):** garantiza que el algoritmo **no tardará más** que un múltiplo constante de $g(n)$ para entradas suficientemente grandes. Es la cota más utilizada en la práctica porque ofrece una garantía de peor caso.
- **Cota inferior $\Omega(g(n))$ (Big-Omega):** garantiza que el algoritmo tardará **al menos** un múltiplo de $g(n)$.
- **Cota ajustada $\Theta(g(n))$ (Big-Theta):** indica que el algoritmo tarda **exactamente** tanto como $g(n)$, estando acotado superior e inferiormente por la misma clase asintótica.

Cota asintótica no es sinónimo de caso analizado. La notación $O(g(n))$ expresa una cota superior matemática sobre la función que se esté evaluando. Esa función puede ser el coste en el peor caso, en el caso promedio o en el mejor caso, según se declare explícitamente.

En ingeniería computacional se analiza prioritariamente el peor caso porque proporciona una garantía sobre el tiempo máximo, pero la notación Big- O y el "peor caso" son conceptos independientes: primero se fija el escenario o caso a estudiar (Sección ??) y posteriormente se acota asintóticamente su función de coste con O , Ω o Θ .

$O(g(n))$ representa una cota superior asintótica (la función no crece más rápido que $g(n)$ salvo constante); $\Omega(g(n))$ representa una cota inferior (crece al menos como $g(n)$); y $\Theta(g(n))$ representa una cota ajustada exacta. La notación de cota es independiente de si se analiza el peor caso, el caso promedio o el mejor caso.

1.3.4 Peor caso, mejor caso y caso promedio

El rendimiento de un algoritmo puede variar no solo según el tamaño n de la entrada, sino según la **disposición de los datos**:

- **Mejor caso:** la configuración más favorable de los datos (por ejemplo, buscar un elemento y que esté en la primera posición: $O(1)$).
- **Peor caso:** la configuración más desfavorable (el elemento buscado está al final o no existe: $O(n)$). Es la métrica estándar de garantía de ingeniería.
- **Caso promedio:** el comportamiento esperado sobre entradas aleatorias o realistas.

Se distinguen peor caso, caso medio y mejor caso de un algoritmo; el análisis de coste se aplica tanto al tiempo como al espacio adicional que necesita.

1.3.5 Clasificación de patrones elementales

Un bucle simple sobre una entrada de tamaño n es $O(n)$; un bucle anidado sobre la misma entrada es $O(n^2)$; un bucle que reduce n a la mitad en cada paso hasta llegar a 1 hace del orden de $\log_2 n$ pasos, $O(\log n)$. Se reproduce con `verification/chapter_results.sh` `classify_single`, `classify_nested` y `classify_halving`.

1.3.6 Demostración matemática del término dominante

Una duda frecuente al aprender notación asintótica es qué significa exactamente que el término $3n^2$ sea «dominante» en la expresión $f(n) = 3n^2 + 500n + 200$.

Analicemos la proporción que representa el término cuadrático $3n^2$ sobre el total:

- Para $n = 10$: $3(10^2) = 300$, frente a un total de $300 + 5000 + 200 = 5500$ (representa solo el 5.4%).
- Para $n = 1000$: $3(1000^2) = 3\,000\,000$, frente a un total de $3\,500\,200$ (representa el **85.7%**).
- Para $n = 100\,000$: $3(100\,000^2) = 30\,000\,000\,000$, frente a $30\,050\,000\,200$ (representa el **99.83%**).

A medida que n crece, el término de mayor exponente explica casi el 100% del coste computacional. Por ello, la regla formal descarta los términos de orden inferior ($500n + 200$) y posteriormente la constante multiplicativa (3), quedando formalmente como $O(n^2)$.

Para n suficientemente grande, el término dominante completo, $3n^2$ (con su constante), representa la práctica totalidad del valor de $3n^2 + 500n + 200$ —85,7% en $n = 1000$, 99,8% en $n = 100\,000$ —; la notación O correcta es $O(n^2)$, no « $O(3n^2 + 500n)$ », porque una vez

identificado el término dominante completo se descartan tanto los términos de orden inferior como su constante multiplicativa. Se reproduce con `verification/chapter_results.sh` `dominant_term_share`.

1.4 Las cinco grandes familias de diseño algorítmico

Ante un problema computacional nuevo en bioinformática, los científicos no reinventan la rueda desde cero: aplican uno de los cinco grandes **paradigmas de diseño algorítmico**.

1.4.1 1. Fuerza bruta y búsqueda exhaustiva

La estrategia de **fuerza bruta** genera y comprueba sistemáticamente todas las soluciones posibles del espacio de búsqueda hasta encontrar la correcta o la óptima. Garantiza siempre encontrar la respuesta correcta si existe, pero su coste suele escalar de forma exponencial ($O(2^n)$) o factorial ($O(n!)$), volviéndose impracticable para entradas reales.

La fuerza bruta explora todas las soluciones posibles de un problema y garantiza encontrar la óptima, a costa de un coste típicamente $O(2^n)$ o $O(n!)$ que la hace inviable salvo para entradas pequeñas.

Caso resuelto: Búsqueda de sitios de corte de restricción. Consideremos la tarea de localizar todas las dianas de corte de la enzima de restricción *EcoRI* (motivo GAATTC, de longitud $k = 6$) en una secuencia de ADN de longitud n . La fuerza bruta compara el motivo contra todas las $n - k + 1$ posiciones posibles de la secuencia.

Listing 1.2: Fuerza bruta: comparar el patrón contra cada posición, sin descartar ninguna.

```
funcion mapaDeRestriccion(secuencia, patron):
  posiciones <- []
  n <- longitud(secuencia)
  k <- longitud(patron)
  para i desde 0 hasta n-k: # las n-k+1 posiciones posibles
    si secuencia[i .. i+k-1] == patron:
      agregar i a posiciones
  devolver posiciones # ninguna posicion se descarta a priori
```

Buscar por fuerza bruta las apariciones de una subcadena de longitud k en una secuencia de longitud n compara la subcadena contra las $n - k + 1$ posiciones posibles: $O(n \cdot k)$ en el peor caso, una por comparación de hasta k caracteres en cada posición. Se reproduce con `verification/chapter_results.sh` `restriction_sites`.

1.4.2 2. Algoritmos voraces (*Greedy*)

Un algoritmo **voraz** construye la solución paso a paso, tomando en cada etapa la decisión que parece **óptima localmente**, con la esperanza de alcanzar el óptimo global. Los algoritmos voraces son extraordinariamente rápidos (típicamente $O(n)$ o $O(n \log n)$), pero tienen una limitación crítica: **no siempre garantizan el óptimo global**.

Un algoritmo voraz toma en cada paso la decisión que parece mejor localmente; solo es correcto si se demuestra que las decisiones locales óptimas construyen la solución global óptima. «Parece óptimo» no es una demostración.

Listing 1.3: Voraz: en cada paso, la moneda más grande que quepa, sin reconsiderar.

```
funcion cambioVoraz(objetivo, monedas_descendente):
  resultado <- []
  restante <- objetivo
```

```

para cada moneda en monedas_descendente: # de mayor a menor valor
  mientras moneda <= restante:
    agregar moneda a resultado
    restante <- restante - moneda # decision local, nunca se revisa
devolver resultado

```

El contraejemplo del cambio de monedas. Supongamos que debemos devolver un cambio de **6 unidades** disponiendo de monedas con valores $\{1, 3, 4\}$.

- **Estrategia voraz:** Toma la moneda más grande posible (4), quedando 2 por cubrir. No cabe otra de 4 ni de 3, por lo que toma una de 1 (queda 1) y otra de 1 (queda 0). Resultado: **3 monedas** ($4 + 1 + 1$).
- **Solución óptima real:** Tomar dos monedas de 3. Resultado: **2 monedas** ($3 + 3$).

El algoritmo voraz falló porque la elección local «tomar la moneda de 4» bloqueó la combinación global óptima ($3 + 3$).

Con monedas de valor $\{1, 3, 4\}$ y un cambio de 6, el método voraz (tomar siempre la moneda más grande que quepa) usa 3 monedas ($4 + 1 + 1$), mientras que la solución óptima usa 2 ($3 + 3$): el voraz no alcanza el óptimo en este caso. Se reproduce con `verification/chapter_results.sh greedy_change_1_3_4`.

Listing 1.4: Algoritmo formal en pseudocódigo para la selección voraz de fragmentos de restricción disjuntos.

```

ALGORITMO: Seleccion_Voraz_Fragmentos_Restriccion
ENTRADA:
  - F: Lista de n fragmentos genomicos, donde cada fragmento i tiene (inicio_i,
    fin_i) con inicio_i < fin_i
SALIDA:
  - S: Subconjunto disjunto maximo de fragmentos no solapantes
COMPLEJIDAD: Tiempo  $O(n \log n)$  por la ordenacion inicial, Espacio auxiliar  $O(n)$ 

1. INICIALIZACION:
  # Criterio voraz optimo: ordenar los fragmentos por tiempo/posicion de
    finalizacion ascendente
  F_ordenado <- Ordenar(F, clave=fin ascendente)
  S <- Conjunto_Vacio()
  ultimo_fin <- -INFINITO

2. BUCLE VORAZ:
  Para cada fragmento f = (inicio, fin) en F_ordenado:
    Si f.inicio >= ultimo_fin entonces:
      # El fragmento es compatible con los seleccionados previamente
      Agregar f a S
      ultimo_fin <- f.fin # Decision irrevocable localmente optima

3. TERMINACION Y RETORNO:
  Retornar S (Se demuestra que |S| alcanza el maximo teorico global)

```

Problema general frente a instancia. Con las denominaciones del sistema monetario del euro (1, 2, 5, 10, 20, 50 céntimos, 1, 2 euros), el método voraz siempre produce el óptimo global porque el sistema es canónico. Esto ilustra un principio fundamental: la validez de un algoritmo voraz depende de la **instancia concreta** de los datos.

Un algoritmo se define para un problema general; una instancia es el caso concreto sobre el que opera (por ejemplo, qué denominaciones de moneda hay disponibles). El mismo algoritmo voraz de cambio de moneda es óptimo para las monedas del sistema euro pero no para $\{1, 3, 4\}$: la corrección de un voraz depende de la instancia, no solo del problema.

1.4.3 3. Divide y vencerás y recursión

El paradigma de **divide y vencerás** descompone un problema en subproblemas más pequeños del mismo tipo, resuelve los subproblemas de forma independiente mediante llamadas recursivas y finalmente **combina** sus soluciones para resolver el problema original.

Una solución recursiva resuelve un problema reduciéndolo a versiones más pequeñas de sí mismo; divide y vencerás parte el problema en subproblemas independientes, los resuelve recursivamente y combina sus resultados.

Anatomía de la recursión: Toda función recursiva correcta debe poseer dos elementos indispensables:

1. **Caso base:** la condición de parada simple que devuelve un resultado directo sin realizar llamadas adicionales.
2. **Llamada recursiva:** la invocación a sí misma que reduce el tamaño del problema acercándose estrictamente al caso base.

Listing 1.5: Fibonacci recursivo sin memoizar.

```
fib(n):
    si n <= 1: devolver n
    devolver fib(n-1) + fib(n-2)
```

```
fib(4)
+-- fib(3)
|   +-- fib(2)
|   |   +-- fib(1)
|   |   '-- fib(0)
|   '-- fib(1)
'-- fib(2)      <- recalculado desde cero
    +-- fib(1)
    '-- fib(0)
```

Figure 1.2: Árbol de llamadas de `fib(4)`: `fib(2)` se calcula dos veces de forma independiente, cada vez repitiendo su propio subárbol completo. Figura original. Texto alternativo: un árbol de llamadas muestra que `fib(4)` llama a `fib(3)` y `fib(2)`; `fib(3)` vuelve a llamar a `fib(2)`, que se recalcula por completo una segunda vez desde cero.

En el árbol de llamadas de `fib(5)` sin memoizar, `fib(2)` se calcula 3 veces de forma independiente; el número total de llamadas crece de forma exponencial, $O(2^n)$. Se reproduce con `verification/chapter_results.sh fib_naive_calls`.

Peligro de desbordamiento de pila (*Stack Overflow*): Cada llamada a una función consume memoria en la pila del sistema (*stack*) para almacenar variables locales y la dirección de retorno. Si falta el caso base, la recursión entrará en un bucle infinito que agotará la memoria de la pila, provocando el colapso inmediato del programa.

Toda solución recursiva requiere un caso base (la condición simple que detiene la recursión sin más llamadas) y una llamada recursiva que reduzca el problema hacia él; sin un caso base alcanzable, cada llamada consume memoria de pila adicional sin límite hasta agotarla (desbordamiento de pila).

La iteración usa menos memoria que la recursión porque no acumula llamadas en la pila; la recursión es más natural para problemas de estructura repetitiva o divisible, como recorrer árboles y grafos o aplicar divide y vencerás, a costa de esa memoria adicional por cada llamada abierta.

1.4.4 4. Programación dinámica

La **programación dinámica** es una de las técnicas más potentes de la computación científica y la base de los algoritmos de alineamiento de secuencias biológicas (como Needleman-Wunsch y Smith-Waterman). Se aplica cuando un problema cumple dos propiedades:

1. **Subestructura óptima:** la solución óptima del problema global se construye a partir de las soluciones óptimas de sus subproblemas.
2. **Subproblemas solapados:** los mismos subproblemas aparecen y se necesitan una y otra vez durante el cálculo.

La idea maestra es la **memoización**: calcular la solución de cada subproblema exactamente una sola vez y guardarla en una tabla o matriz para reutilizarla de inmediato en tiempo constante ($O(1)$).

La programación dinámica se aplica cuando un problema tiene subestructura óptima y subproblemas solapados; memoizar (guardar el resultado de cada subproblema la primera vez que se calcula) evita recalcularlo, convirtiendo una recursión exponencial en un algoritmo eficiente.

Memoizando `fib`, cada valor de 0 a n se calcula exactamente una vez: el número de cálculos pasa de exponencial a $O(n)$. Se reproduce con `verification/chapter_results.sh fib_memo_calls`.

Listing 1.6: Algoritmo formal en pseudocódigo comparando Fibonacci ingenuo vs Programación Dinámica memoizada.

```
ALGORITMO: Fibonacci_Comparativa_Complejidad
ENTRADA:
- n: Entero no negativo representando la posicion en la serie de Fibonacci
SALIDA:
- F_n: El n-esimo numero de Fibonacci

# VARIANTE 1: INGENUA RECURSIVA (Inviabile: Tiempo  $O(2^n)$ , Espacio pila  $O(n)$ )
Funcion Fibonacci_Ingenuo(n):
  Si n <= 0 entonces retornar 0
  Si n = 1 entonces retornar 1
  Retornar Fibonacci_Ingenuo(n - 1) + Fibonacci_Ingenuo(n - 2)

# VARIANTE 2: PROGRAMACION DINAMICA MEMOIZADA (Optima: Tiempo  $O(n)$ , Espacio  $O(1)$ )
ALGORITMO: Fibonacci_DP_Iterativo_Optimo(n):
  Si n <= 0 entonces retornar 0
  Si n = 1 entonces retornar 1

  prev2 <- 0 # F_{i-2}
  prev1 <- 1 # F_{i-1}

  Para i desde 2 hasta n:
```

```

    actual <- prev1 + prev2 # F_i = F_{i-1} + F_{i-2}
    prev2 <- prev1
    prev1 <- actual

```

Retornar actual # Calculado en exactamente (n-1) sumas elementales

Resolución del cambio de monedas con programación dinámica: Definiendo $dp[c]$ como el número mínimo de monedas para formar el importe c , con $dp[0] = 0$:

$$dp[c] = 1 + \min_{\substack{m \in \{1,3,4\} \\ m \leq c}} dp[c - m]$$

Listing 1.7: Programación dinámica ascendente (*bottom-up*): resuelve todos los importes menores antes que el pedido.

```

funcion monedasMinimasDP(objetivo, monedas):
    dp[0] <- 0
    para c desde 1 hasta objetivo:
        dp[c] <- infinito
        para cada m en monedas:
            si m <= c:
                dp[c] <- min(dp[c], dp[c - m] + 1)
    devolver dp[objetivo]

```

El cambio de monedas con $\{1, 3, 4\}$ tiene subestructura óptima y subproblemas solapados (por ejemplo, $dp[3]$ se necesita para calcular $dp[4]$ y $dp[6]$); se resuelve con la relación de recurrencia $dp[c] = 1 + \min_{m \leq c} dp[c - m]$ con $dp[0] = 0$.

Sobre monedas $\{1, 3, 4\}$ y objetivo 6, la tabla de programación dinámica da $dp[0..6] = 0, 1, 2, 1, 1, 2, 2$: el importe 6 se resuelve con 2 monedas ($dp[6] = 2$, correspondiente a $3 + 3$), el óptimo real, no las 3 monedas ($4 + 1 + 1$) que dio el algoritmo voraz sobre la misma instancia. La programación dinámica considera todas las combinaciones posibles a través de la tabla, sin descartar ninguna opción localmente antes de tiempo. Se reproduce con `verification/chapter_results.sh coin_change_dp 6`.

Memoizar solo aporta cuando existen subproblemas solapados; en un algoritmo cuyos subproblemas son independientes y no se repiten (como la ordenación por mezcla), memoizar no ahorra cómputo y solo añade sobrecarga de memoria.

1.4.5 5. Vuelta atrás (*Backtracking*)

El paradigma de **vuelta atrás** (***backtracking***) es una optimización de la fuerza bruta para problemas de búsqueda combinatoria. En lugar de generar todas las combinaciones posibles y filtrarlas al final, el algoritmo construye soluciones parciales candidatas y **abandona inmediatamente** (***poda***) una rama en cuanto comprueba que no puede satisfacer las restricciones del problema.

Listing 1.8: Plantilla general de backtracking.

```

backtracking(solucion_parcial, datos_restantes):
    si solucion_parcial es completa:
        si cumple los criterios: guardarla
        devolver
    para cada opcion posible en datos_restantes:
        agregar opcion a solucion_parcial
        backtracking(solucion_parcial, datos_restantes sin opcion)
        quitar opcion de solucion_parcial # deshacer: el paso que no tiene la fuerza
        bruta

```

Backtracking explora el espacio de soluciones abandonando una rama en cuanto se comprueba que no puede llevar a una solución válida, a diferencia de la fuerza bruta, que completa cada combinación antes de descartarla; su implementación típica se apoya en una pila que guarda el camino parcial.

Caso resuelto: Generación de motivos de ADN sin pares AA. Supongamos que deseamos generar todas las secuencias de longitud $L = 4$ sobre el alfabeto $\{A, T\}$ con la restricción biológica de que nunca aparezcan dos adeninas consecutivas (AA).

- **Fuerza bruta sin podar:** Exploraría un árbol completo de $2^0 + 2^1 + 2^2 + 2^3 + 2^4 = 31$ nodos para generar 16 secuencias y descartar las inválidas.
- **Backtracking con poda:** En cuanto añade una A y la siguiente candidata es otra A, aborta la rama inmediatamente. Solo visita **19 nodos** y devuelve directamente las **8 cadenas válidas**.

Para generar cadenas de longitud 4 sobre $\{A, T\}$ sin dos "A" consecutivas, backtracking visita 19 nodos del árbol de búsqueda, frente a los 31 nodos que tendría el árbol completo sin podar ($2^0 + 2^1 + \dots + 2^4$); encuentra exactamente 8 cadenas válidas. Se reproduce con `verification/chapter_results.sh backtracking_count`.

La predicción de estructura secundaria de ARN por emparejamiento de bases y el alineamiento simultáneo de tres o más secuencias son dos problemas que se pueden plantear como backtracking con poda; ambos son desarrollos de capítulos posteriores y no se implementan aquí.

1.5 La operación peligrosa: la explosión combinatoria oculta

En bioinformática, una de las trampas más devastadoras ocurre cuando un algoritmo con complejidad exponencial $O(2^n)$ se programa y prueba con entradas de juguete ($n = 10$) durante la fase de desarrollo. El algoritmo parece funcionar de forma instantánea ($2^{10} = 1024$ operaciones). Sin embargo, al aplicarlo al conjunto de datos real de laboratorio con $n = 50$, el número de operaciones salta a $2^{50} \approx 1.12 \times 10^{15}$, provocando el bloqueo total del servidor, saturación de la memoria RAM y pérdida de tiempo de computación.

Protocolo preventivo en 3 pasos ante la escala de datos:

1. **Análisis asintótico previo:** deducir la cota Big-O formal del algoritmo antes de escribir la primera línea de código.
2. **Prueba de escalado con duplicación ($2\times$):** ejecutar el algoritmo con entradas de tamaño creciente ($n = 10, 20, 30, 40$) midiendo el tiempo de ejecución. Si al duplicar n el tiempo se multiplica por 4, es cuadrático ($O(n^2)$); si se eleva exponencialmente, es $O(2^n)$.
3. **Validación de límites y guardia de tamaño:** establecer una comprobación preventiva en el código que rechace o alerte ante tamaños de entrada que excedan la capacidad computacional disponible.

1.6 Actividad de depuración: localizar fallos en algoritmos ajenos

Diagnosticar errores en algoritmos diseñados por otros o por herramientas de IA es una habilidad crítica. Analice el siguiente fragmento con tres errores conceptuales:

Listing 1.9: Algoritmo con tres fallos conceptuales para depurar.

```

funcion buscarRepetidos(lista, n):
    # Fallo 1: inicializacion incorrecta de cota
    para i desde 0 hasta n:
        # Fallo 2: comparacion redundante cuadratica innecesaria
        para j desde 0 hasta n:
            si i != j y lista[i] == lista[j]:
                devolver VERDADERO
    devolver FALSO

```

Diagnóstico de los fallos:

1. **Desbordamiento de índice:** el bucle va hasta n en lugar de $n - 1$, accediendo a una posición fuera de los límites del vector.
2. **Redundancia de comparaciones cuadráticas:** al iterar j desde 0 hasta n , cada par (i, j) se compara dos veces. Iterando j desde $i + 1$ hasta $n - 1$ se reduce el número de operaciones exactamente a la mitad.
3. **Ineficiencia de paradigma:** si la lista se ordena previamente con Mergesort ($O(n \log n)$), los elementos duplicados quedarán adyacentes y podrán detectarse en una sola pasada lineal ($O(n)$), reduciendo el coste total de $O(n^2)$ a $O(n \log n)$.

1.7 Síntesis y diagnóstico de problemas algorítmicos

Técnica	Idea central	¿Óptimo?	Coste típico	Ejemplo en biología
Fuerza bruta	Probar todas las opciones	Sí	$O(2^n)$, $O(n!)$	Búsqueda exhaustiva de diana
Voraz (*Greedy*)	Elegir lo mejor local	No siempre	$O(n)$, $O(n \log n)$	Selección de experimentos
Divide y vencerás	Dividir y combinar	Depende	$O(n \log n)$	Mergesort en secuencias
Prog. Dinámica	Subproblemas solapados	Sí	$O(n^2)$, $O(n^3)$	Alineamiento de secuencias
Backtracking	Podar ramas inviables	Sí	Variable ($< 2^n$)	Plegamiento de ARN

Las cinco técnicas de diseño se sintetizan por idea central, optimalidad garantizada, coste típico y un ejemplo biológico: fuerza bruta (probar todas las soluciones, siempre óptima, $O(2^n)/O(n!)$, búsqueda exhaustiva de subcadena); voraz (elegir lo mejor en cada paso, no siempre óptimo, $O(n)/O(n \log n)$, selección de actividades); divide y vencerás (partir y combinar, depende, coste variable, árbol filogenético); programación dinámica (subproblemas solapados más memoización, óptima, $O(n^2)/O(n^3)$, alineamiento de secuencias); backtracking (podar ramas inválidas pronto, óptima, coste variable pero mejor que fuerza bruta, ensamblaje combinatorio restringido).

1.7.1 Tabla de diagnóstico de síntomas computacionales

Síntoma observado	Causa probable	Comprobación y corrección
El programa se congela al aumentar N de 15 a 30	Algoritmo con complejidad exponencial $O(2^n)$	Comprobar si hay subproblemas solapados y aplicar memoización
Error <i>Stack overflow</i> / desbordamiento	Recursión sin caso base o llamada que no reduce N	Verificar que el caso base esté al principio y sea alcanzable
La solución voraz devuelve un resultado subóptimo	La instancia no cumple la propiedad voraz	Plantear la relación de recurrencia con Programación Dinámica
El tiempo se multiplica por 4 al duplicar N	Bucle anidado oculto ($O(n^2)$)	Ordenar previamente ($O(n \log n)$) o usar indexación hash ($O(1)$)

1.7.2 Tabla de síntesis operativa con preguntas de control

Necesidad	Paradigma recomendado	Pregunta de control
Probar todo el espacio de soluciones	Fuerza bruta	¿Es n suficientemente pequeño ($n \leq 15$)?
Solución rápida con elecciones locales	Algoritmo voraz (*Greedy*)	¿Está demostrado que la elección local lleva al óptimo global?
Subproblemas independientes	Divide y vencerás	¿Cómo se combinan las soluciones parciales?
Subproblemas repetidos y solapados	Programación dinámica	¿Tengo memoria suficiente para la tabla de memoización?
Espacio combinatorio con restricciones	Backtracking con poda	¿Puedo detectar y abortar ramas inválidas en etapas tempranas?

1.8 Glosario esencial

- **Algoritmo:** secuencia finita y bien definida de pasos para resolver un problema computacional.
- **Notación Big-O (O):** cota asintótica superior que describe el orden de crecimiento del coste de un algoritmo.
- **Modelo RAM:** abstracción de computador donde las operaciones elementales de CPU y memoria toman tiempo unitario constante.
- **Fuerza bruta:** exploración exhaustiva de todas las soluciones posibles.
- **Algoritmo voraz (*Greedy*):** método que selecciona la opción óptima local en cada paso.
- **Divide y vencerás:** técnica que divide un problema en partes independientes, las resuelve y combina los resultados.
- **Recursión:** mecanismo por el cual una función se invoca a sí misma sobre una instancia reducida.
- **Caso base:** condición de terminación en una función recursiva.
- **Programación dinámica:** técnica para problemas con subestructura óptima y subproblemas solapados mediante memoización.
- **Memoización:** almacenamiento de resultados intermedios en una tabla para evitar recálculos.
- **Backtracking:** exploración combinatoria con retroceso y poda temprana de ramas inválidas.

1.9 Conexión con el curso y lo que viene después

Este capítulo constituye la piedra angular metodológica de toda la bioinformática algorítmica que desarrollaremos en los siguientes temas:

- En **TC-CH04** y **TC-CH05**, formalizaremos las estructuras de datos fundamentales (lineales, árboles, tablas hash y grafos biológicos) analizando el coste temporal y espacial de cada operación de acceso y mutación.
- En **TC-CH06**, aplicaremos y compararemos sistemáticamente algoritmos de búsqueda y ordenación (lineal, binaria, QuickSort, Mergesort, Heapsort y Radix Sort), comprobando empíricamente cotas y constantes.
- En **TC-CH07** y **TC-CH08**, aplicaremos teoría de la información y modelos probabilísticos (cadenas de Markov y HMMs con el algoritmo de Viterbi).
- En **TC-CH09**, la programación dinámica resolverá el alineamiento de secuencias con Needleman–Wunsch y Smith–Waterman, y la heurística de BLAST permitirá explorar bases de datos a escala genómica.
- En **TC-CH10**, extendaremos el aprendizaje computacional hacia redes neuronales y representaciones aprendidas.

La formalización de complejidad asintótica y paradigmas de este capítulo fundamenta el análisis de estructuras en TC-CH04/05, la búsqueda y ordenación en TC-CH06, los modelos probabilísticos en TC-CH08, el alineamiento en TC-CH09 y el aprendizaje profundo en TC-CH10.

1.10 Fuentes y límites de uso

Este capítulo se fundamenta en la presentación docente oficial de la asignatura (20250930_alg_1.pptx, Álvaro Serrano Navarro), en el texto clásico de algoritmos *Introduction to Algorithms* (Cormen et al., MIT Press) y en la literatura bioinformática de referencia (*Bioinformatics Algorithms*, Compeau & Pevzner).

Anexo práctico · Auditar el coste y sentir la diferencia entre exponencial y lineal

Pregunta, producto y separación de materiales

¿Puede un estudiante de biociencias auditar tres afirmaciones de coste erróneas producidas por una IA y medir, con sus propias manos en la terminal, la diferencia real entre una recursión desbocada y una solución memoizada? Este laboratorio responde afirmativamente sin requerir ningún dato externo: todo se genera y se mide con `verification/chapter_results.sh`.

El producto individual contiene: un veredicto razonado sobre las tres afirmaciones de coste auditadas, la tabla de mediciones de `fib` ingenuo frente a memoizado, la comprobación de escalado al duplicar la entrada, y las dos comprobaciones de control, todo en `notas/respuestas.tsv`.

El anexo es autónomo e independiente: no comparte datos con otros capítulos. Utiliza `practice_assets/create_workspace.sh` para generar el espacio de trabajo del alumno y `verification/chapter_results.sh` como herramienta de cómputo y auditoría.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Preparar espacio de trabajo	5	Directorio creado, herramienta comprobada
2 · Auditar afirmaciones de coste	20–25	Veredicto y justificación por fragmento
3 · Medir <code>fib</code> ingenuo vs memoizado	20–25	Tabla de mediciones + interpretación
4 · Perturbación: duplicar entrada	15–20	Cociente observado vs esperado ($2\times$)
5 · Controles de robustez	10–15	Dos entradas inválidas rechazadas
6 · Verificación y empaquetado	5	Comprobación con <code>DELIVERY_OK</code>

La ruta única de este anexo (no hay separación esencial/estudio) suma 70–100 minutos y produce evidencia comprobable en cada bloque.

1 · Preparar el espacio de trabajo

Listing 1.10: Comprobación de entorno y espacio de trabajo.

```
bash --version
chmod +x practice_assets/create_workspace.sh
./practice_assets/create_workspace.sh TC03_entrega
cd TC03_entrega
```

`create_workspace.sh` genera las carpetas `notas/` y `resultados/`, copiando `verification/chapter_results.sh` en `herramienta/` para que las pruebas se ejecuten de forma aislada.

2 · Auditar tres afirmaciones de coste generadas por IA

Un asistente de IA clasifica tres fragmentos de código con errores: (a) un bucle anidado que compara cada elemento de una secuencia con todos los demás lo clasifica como « $O(n)$ »; (b) un bucle que reduce n a la mitad en cada paso hasta llegar a 1 lo clasifica como « $O(n)$ »; (c) un bucle simple que recorre una secuencia una sola vez lo clasifica como « $O(n^2)$ ».

El fragmento (a) —bucle anidado— es en realidad $O(n^2)$, no $O(n)$; el fragmento (b) —reducir n a la mitad— es en realidad $O(\log n)$, no $O(n)$; y el fragmento (c) —bucle simple— es en realidad $O(n)$, no $O(n^2)$: las tres clasificaciones propuestas están intercambiadas o infladas respecto al orden real.

Listing 1.11: Medir el coste real de los tres fragmentos.

```
bash herramienta/chapter_results.sh classify_nested 10 # fragmento (a)
bash herramienta/chapter_results.sh classify_halving 1000000 # fragmento (b)
bash herramienta/chapter_results.sh classify_single 10 # fragmento (c)
```

Para cada fragmento, registre en `notas/respuestas.tsv` las columnas `fragmento`, `veredicto_ia`, `orden_real` y `justificacion`.

3 · Medir la programación dinámica: Fibonacci

Listing 1.12: Comparar fib sin memoizar frente a memoizado.

```
bash herramienta/chapter_results.sh fib_naive_calls 5
bash herramienta/chapter_results.sh fib_memo_calls 5
bash herramienta/chapter_results.sh time_compare_verdict 15
time bash herramienta/chapter_results.sh fib_naive_calls 18
```

Para el mismo valor de n , el tiempo real que tarda `fib_naive` es mayor que el que tarda la tabla ascendente de `fib_memo_calls`; la diferencia crece con n porque una crece exponencialmente y la otra linealmente. Se reproduce con `verification/chapter_results.sh time_compare_verdict`.

Antes de ejecutar la última orden (`time ... fib_naive_calls 18`), **prediga por escrito** si tardará menos de un segundo o varios segundos, justificando el motivo según la tasa $O(2^n)$. Compare con la salida de `time`.

4 · Perturbación: duplicar el tamaño de la entrada ($2\times$)

Listing 1.13: Duplicar N y observar el cociente de operaciones.

```
bash herramienta/chapter_results.sh classify_single 10
bash herramienta/chapter_results.sh classify_single 20
bash herramienta/chapter_results.sh classify_nested 10
bash herramienta/chapter_results.sh classify_nested 20
```

Al duplicar n de 10 a 20, las operaciones del fragmento $O(n)$ crecen aproximadamente $\times 2$ (de 51 a 101), mientras que las del fragmento $O(n^2)$ crecen aproximadamente $\times 4$ (de 55 a 210): el cociente observado coincide con lo que predice cada clase de la notación O . Se reproduce con `verification/chapter_results.sh classify_single` y `classify_nested` sobre $n = 10$ y $n = 20$.

5 · Controles de robustez y validación de límites

Listing 1.14: Comprobar el rechazo de entradas inválidas.

```
bash herramienta/chapter_results.sh control_reject_invalid -3
bash herramienta/chapter_results.sh control_reject_invalid abc
```

`fib_naive_calls` y `fib_memo_calls` rechazan explícitamente una entrada negativa o no numérica en vez de fallar de forma opaca en la aritmética de Bash o entrar en un comportamiento no definido. Se reproduce con `verification/chapter_results.sh control_reject_invalid`.

6 · Verificación automática y empaquetado

Listing 1.15: Comprobar que la entrega está completa antes de empaquetar.

```
cd ..
../practice_assets/check_practice_delivery.sh TC03_entrega
tar -czf TC03_entrega.tar.gz TC03_entrega
sha256sum TC03_entrega.tar.gz
```

`check_practice_delivery.sh` verifica, con Bash y GNU Coreutils exclusivamente, que `notas/respuestas.tsv` existe con la cabecera esperada y al menos una fila rellena, y que `herramienta/chapter_results.sh` está presente y es ejecutable.

Rúbrica de calificación ponderada

La evaluación de este anexo pondera la verificación técnica, el análisis formal y la defensa conceptual sobre 10 puntos (reparto 30/70 según ADR-001):

- **3,0 puntos · Entrega técnica y verificador (DELIVERY_OK):** Estructura de carpetas requerida, formato de `notas/respuestas.tsv` y finalización exitosa de `check_practice_delivery.sh`.
- **2,5 puntos · Auditoría analítica de fragmentos:** Identificación rigurosa del orden asintótico real de los tres fragmentos propuestos y justificación formal anclada en el número de iteraciones y ramificaciones.
- **2,5 puntos · Experimentación y contraste de perturbación (2×):** Medición comparativa de Fibonacci, interpretación del cociente de operaciones al duplicar n (2× para $O(n)$, 4× para $O(n^2)$) y control de entradas inválidas.
- **2,0 puntos · Defensa conceptual y conexión biológica:** Justificación rigurosa (100–150 palabras) sobre el papel de los subproblemas solapados en la memoización frente a subproblemas disjuntos, apoyándose en los datos obtenidos.

Nota didáctica: La obtención de `DELIVERY_OK` certifica la presencia e integridad del paquete de entrega, pero no sustituye la fundamentación teórica de la complejidad ni la defensa conceptual.

Lista de autocorrección

- Las tres afirmaciones de coste auditadas tienen orden real y justificación razonada.
- Comparé mi predicción de tiempo de `fib_naive_calls 18` con la medida real de `time`.

- El cociente calculado al duplicar n concuerda con la teoría ($2\times$ para $O(n)$, $4\times$ para $O(n^2)$).
- Las entradas inválidas fueron rechazadas explícitamente con mensaje de control.
- Puedo explicar por qué memoizar transforma una complejidad exponencial en lineal.
- El script `check_practice_delivery.sh` termina con `DELIVERY_OK`.

Defensa conceptual

En 100–150 palabras, explique por qué la memoización acelera drásticamente el cálculo de `fib` pero no aportaría ninguna ventaja en un algoritmo cuyos subproblemas son estrictamente independientes y nunca se repiten (como el algoritmo de ordenación por mezcla *Mergesort*). Emplee al menos un dato numérico concreto obtenido en sus propias mediciones del Bloque 3.

Fuentes y reproducibilidad

Este anexo se apoya en los ejercicios de análisis algorítmico del repositorio docente y se ejecuta de forma 100% determinista mediante `verification/chapter_results.sh`.

Bibliography

- [1] Stefano Allesina and Madlen Wilmes. *Computing Skills for Biologists: A Toolbox*. Princeton University Press, 2019. Ejemplos contrastivos de algoritmos; no sostiene ninguna afirmación en solitario.
- [2] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. 3rd edition, 2018. Encuadre bioinformático de la programación dinámica como motor del alineamiento, citado hacia TC-CH09.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009. Referencia canónica para la prueba del límite de la ordenación por comparación y la clasificación de paradigmas de diseño; no se reproduce, solo se cita.